

Module 11

Assignment 11A: API Data

Course: CS 85 - PHP Programming

Instructor: Vicky Seno

GitHub repository URL: <https://github.com/sergehall/cs85-php-programming/blob/main/assignments/module11a/README.md>

Document Map

- 1. Main Assignment Questions and Answers
- 2. Course-Aligned Implementation Walkthrough
- 3. Required Browser and Code Evidence
- 4. Advanced Implementation Questions and Answers
- 5. Architecture, Security, and Reliability
- 6. Comparison, Reflection, and Submission Checklist

1. Main Assignment Questions and Answers

The main Assignment 11A is a warm-up for live API integration. A static file imitates a weather API response so the JSON-to-Blade workflow can be learned without network failures, authentication, or provider limits.

What is JSON?

JSON stands for JavaScript Object Notation. It is a lightweight text format for storing and transporting structured data. APIs commonly return JSON because it is readable by people and easy for programming languages to decode.

What does `json_decode()` do?

The PHP `json_decode()` function converts a JSON string into a PHP value. In this assignment, `json_decode($json, true)` converts the weather JSON array into PHP associative arrays that can be accessed with keys such as `$day['high']`.

Why is `true` passed as the second argument?

Passing `true` tells PHP to return JSON objects as associative arrays instead of `stdClass` objects. This matches the required Blade syntax: `$day['day']`, `$day['high']`, `$day['low']`, and `$day['condition']`.

Why is `weather.json` stored in `storage/app/private`?

Laravel 11 and later configure the default local disk to use `storage/app/private`. Therefore `Storage::get('weather.json')` resolves to that private directory. The file is also not directly public through a URL, so Laravel controls when and how it is read.

What does the Storage facade provide?

The Storage facade gives the controller a Laravel-native filesystem API. `Storage::get('weather.json')` reads the file from the configured local disk and returns its contents as a string without hard-coding an absolute filesystem path.

How does the controller pass data to Blade?

`WeatherController` reads and decodes the file, then returns `view('weather.index', ['weather' => $weatherData])`. The `weather` key becomes the `$weather` variable inside `resources/views/weather/index.blade.php`.

How does Blade display the structured records?

The `@foreach ($weather as $day)` directive creates one table row for every forecast record. Blade's escaped `{{ }}` output syntax safely prints the day, high temperature, low temperature, and condition.

Which optional improvements were implemented?

Rainy rows receive blue conditional formatting, sunny rows receive amber formatting, and a query control can sort the days alphabetically. The default view preserves the original Monday-to-Wednesday order.

2. Course Aligned Implementation Walkthrough

Step 1 Static JSON data

The weather dataset contains three records. Each record uses the same four fields, which makes the data predictable and easy to display as table columns.

```
[
  {"day": "Monday", "high": 75, "low": 60, "condition": "Sunny"},
  {"day": "Tuesday", "high": 72, "low": 58, "condition": "Partly Cloudy"},
  {"day": "Wednesday", "high": 68, "low": 55, "condition": "Rain"}
]
```

Step 2 Controller logic

The controller intentionally uses the two-step method from the instructions.

`JSON_THROW_ON_ERROR` strengthens the example by making malformed JSON fail explicitly instead of silently returning null.

```
$json = Storage::get('weather.json');
$weatherData = json_decode($json, true, flags: JSON_THROW_ON_ERROR);

return view('weather.index', [
    'weather' => $weatherData,
]);
```

Step 3 Blade table

The view uses semantic table headings and a Blade loop. Each temperature adds the Fahrenheit symbol, while the condition controls the optional row color.

```
@foreach ($weather as $day)
    <tr>
        <td>{{ $day['day'] }}</td>
```

```

<td>{{ $day['high'] }}°F</td>
<td>{{ $day['low'] }}°F</td>
<td>{{ $day['condition'] }}</td>
</tr>
@endforeach

```

Step 4 Route and result

GET /weather is named assignments.module11a.weather and calls WeatherController@index. The resulting page includes the required forecast table, explanation, optional sort control, and a link to the advanced track.

The screenshot shows a web browser displaying the course-aligned /weather page. The page is titled 'Weekly Weather Forecast' and is part of 'Assignment 11A - API Data'. It features a table with the following data:

DAY	HIGH	LOW	CONDITION
Monday	75°F	60°F	* Sunny
Tuesday	72°F	58°F	☁ Partly Cloudy
Wednesday	68°F	55°F	● Rain

The page also includes a 'View weather table' button and a 'DECODED JSON OUTPUT' section. The assignment title and course-aligned track are clearly visible at the top of the page.

Figure 1. Course-aligned /weather page

What this screenshot should prove

- The required /weather route loads successfully in Laravel.
- All three decoded JSON records appear in the styled table.
- The assignment title and course-aligned track are clearly visible.

3. Main Assignment Screenshot Evidence

STEP-BY-STEP SUMMARY

How the exact assignment works

01

Create the file

weather.json lives on Laravel's default private local disk.

02

Read the JSON

Storage::get('weather.json') returns the file as a string.

03

Decode the string

json_decode(\$json, true) creates an associative PHP array.

04

Render the rows

The controller passes weather to Blade and @foreach creates the table body.

COURSE REQUIREMENT

Clean file organization

Data
storage/app/private/weather.json

Controller
app/Http/Controllers/WeatherController.php

View
resources/views/weather/index.blade.php

Route
GET /weather

COURSE CONCEPTS RETAINED

Everything the grader should find

- 01** A valid static JSON array with three weather records.
- 02** Laravel Storage reading from storage/app/private.
- 03** json_decode() returning associative arrays.
- 04** Controller data passed to weather.index.
- 05** A styled HTML table rendered with Blade @foreach.

PROFESSIONAL EXTENSION

Ready for the live API step

The advanced track keeps the same input → decode → transform → render idea, then adds the engineering needed when the data source is remote and unreliable.

[Open advanced implementation](#)

ASSIGNMENT REFLECTION

JSON is the bridge between stored text and structured PHP data.

The JSON file starts as plain text. json_decode(\$json, true) converts it into associative arrays, which makes each field available with keys such as \$day['high']. The controller prepares the data once, and Blade focuses only on presentation.

Figure 2. Optional sorting and conditional formatting

```

1  [
2  {
3    "day": "Monday", "high": 75, "low": 60, "condition": "Sunny"},
4    "day": "Tuesday", "high": 72, "low": 58, "condition": "Partly Cloudy"},
5    "day": "Wednesday", "high": 68, "low": 55, "condition": "Rain"}
6  ]

```

Figure 3. Static weather.json dataset

4. Main Assignment Code Evidence

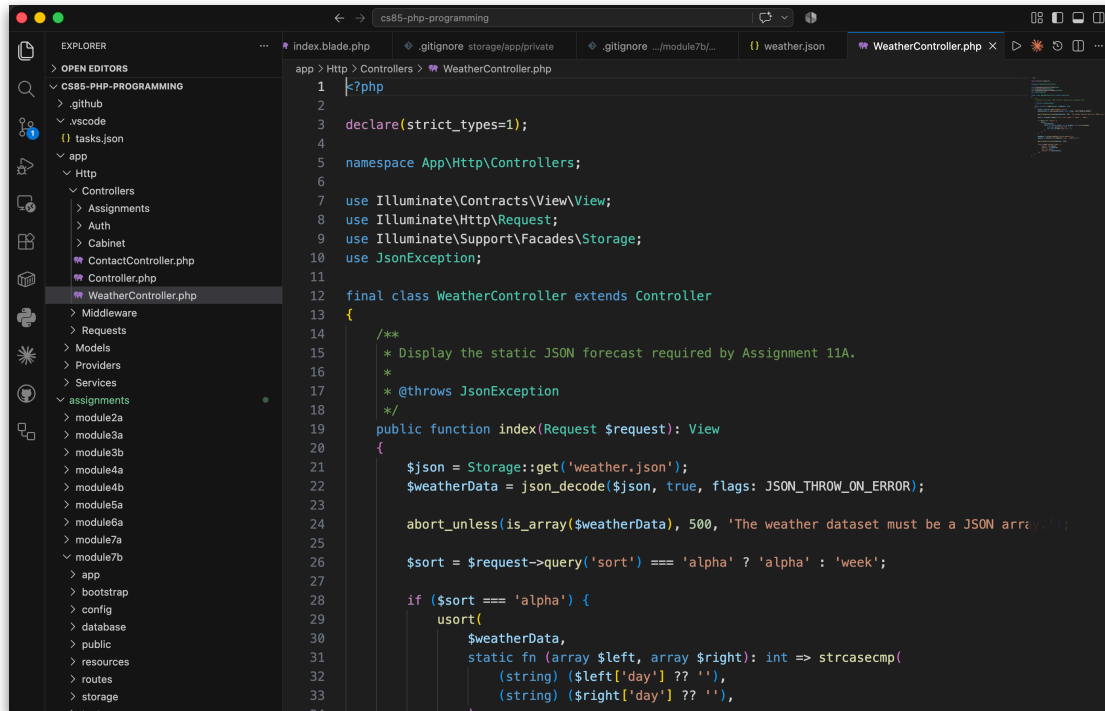


Figure 4. WeatherController JSON decoding

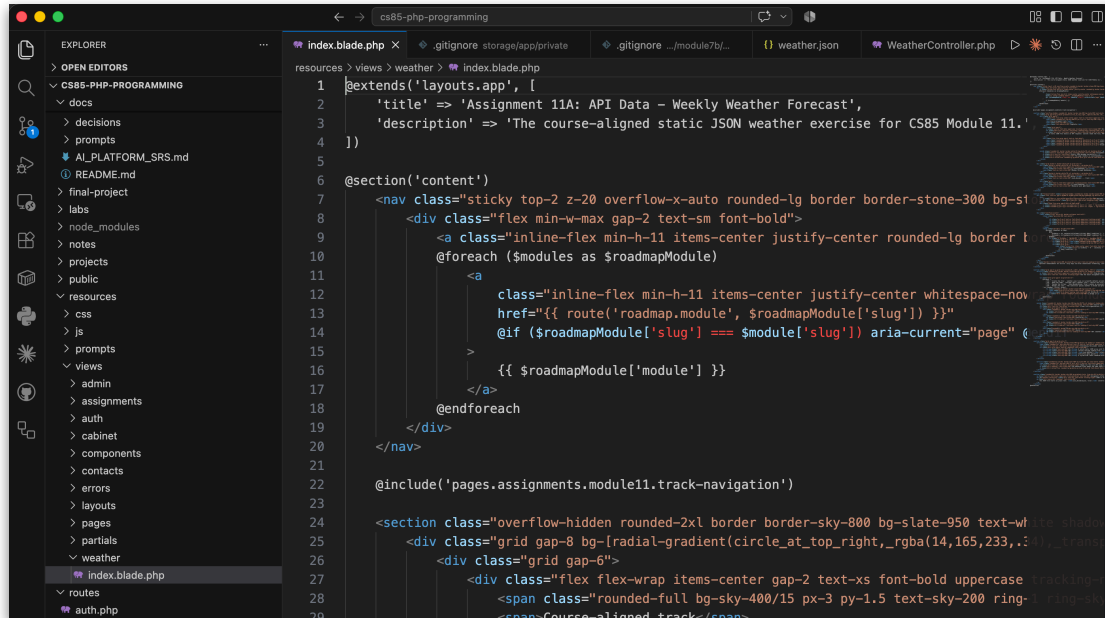


Figure 5. Blade weather table

5. Advanced Implementation Questions and Answers

The advanced track does not replace the required exercise. It demonstrates how the same JSON workflow can be designed for a real third-party provider, where data can be unavailable, malformed, slow, or structurally different.

What does the advanced implementation build?

Laravel fetches users from the JSONPlaceholder API, validates and normalizes each record into an ApiContact object, then exposes searchable contact cards and a normalized JSON endpoint at `/roadmap/module-11/data`.

Why is the API request performed on the server?

Server-side requests keep provider configuration inside Laravel, create one controlled trust boundary, and prevent browser code from depending directly on a third party. This also makes caching, fallback, validation, and testing easier.

Why is the endpoint fixed in configuration?

Allowing visitors to submit an arbitrary server-side URL could create a Server-Side Request Forgery (SSRF) vulnerability. The trusted endpoint is configured under `services.module11a` instead of being accepted from query input.

Why map the response into ApiContact objects?

A provider response is untrusted input, not an application model. ApiContact accepts only complete, valid records and exposes a stable set of fields. The view and JSON endpoint no longer depend on JSONPlaceholder's nested payload.

What is the difference between cache and fallback?

The ten-minute cache reuses a previously validated live response to reduce latency and provider traffic. The fallback is a versioned local dataset used when no usable live response is available. Cache is an optimization; fallback is a resilience mechanism.

How are search and sorting protected?

`BrowseModule11ApiDataRequest` limits search text to 80 characters and allowlists sort and result-limit values. `BrowseApiContacts` applies those operations only to normalized ApiContact objects.

What does the JSON endpoint return?

`GET /roadmap/module-11/data` returns a meta object with source, degraded state, timestamp, counts, and active filters, followed by a data array containing only the normalized contact contract.

Clean Architecture responsibilities

Layer	Implementation	Responsibility
Presentation	Module11ApiController + Blade	Coordinates the request and chooses HTML or JSON output.
Application	BrowseApiContacts	Searches, sorts, limits, and summarizes normalized contacts.
Domain	ApiContact	Defines the trusted typed contract accepted by the application.
Infrastructure	JsonPlaceholderUserClient	Owns HTTP, timeout, cache, response validation, and fallback.

Advanced request lifecycle

1. Validate the allowed search, sort, limit, and refresh query values.
2. Fetch a configured HTTPS endpoint or reuse a validated cached response.
3. Normalize complete provider records into ApiContact objects.
4. Search, sort, limit, and summarize the trusted application data.
5. Render escaped Blade cards or return the normalized JSON contract.

6. Advanced Browser Evidence

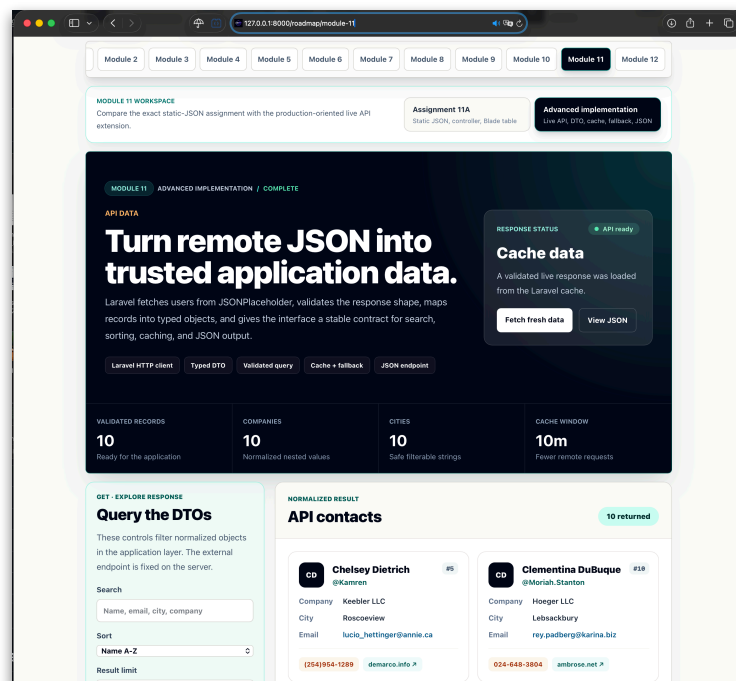


Figure 6. Advanced Module 11 overview

MODULE 11 WORKSPACE

Compare the exact static-JSON assignment with the production-oriented live API extension.

Assignment 11A
Static JSON, controller, Blade table

Advanced Implementation
Live API, DTO, cache, fallback, JSON

MODULE 11 ADVANCED IMPLEMENTATION / COMPLETE

API DATA

Turn remote JSON into trusted application data.

Laravel fetches users from JSONPlaceholder, validates the response shape, maps records into typed objects, and gives the interface a stable contract for search, sorting, caching, and JSON output.

RESPONSE STATUS: API ready

Cache data

A validated live response was loaded from the Laravel cache.

Fetch fresh data View JSON

Laravel HTTP client Typed DTO Validated query Cache + fallback JSON endpoint

VALIDATED RECORDS
10
Ready for the application

COMPANIES
10
Normalized nested values

CITIES
10
Safe filterable strings

CACHE WINDOW
10m
Fewer remote requests

GET - EXPLORE RESPONSE

Query the DTOs

These controls filter normalized objects in the application layer. The external endpoint is fixed on the server.

Search: Patricia Lebsack

Sort: Name A-Z

Result limit: 10 records

Run query

NORMALIZED RESULT

API contacts 1 returned

Patricia Lebsack #4
@Karianne

Company: Robel-Corkery
City: South Elvis
Email: julianne.oconner@kory.org

493-170-9623 x156 kale.biz

Figure 7. Validated API search result

```
{
  "meta": {
    "source": "cache",
    "degraded": false,
    "fetched_at": "2026-07-27T22:30:57+00:00",
    "total": 10,
    "returned": 1,
    "filters": {
      "search": "Gwenborough",
      "sort": "name_asc",
      "limit": 4
    }
  },
  "data": [
    {
      "id": 1,
      "name": "Leanne Graham",
      "username": "Bret",
      "email": "sincere@april.biz",
      "phone": "1-778-736-8031 x56442",
      "website": "hildegard.org",
      "company": "Romaguera-Crona",
      "city": "Gwenborough"
    }
  ]
}
```

Figure 8. Normalized JSON endpoint

Reliability and security summary

- HTTPS provider endpoint with Accept: application/json.
- Short configurable timeout so the page does not wait indefinitely.
- DTO validation rejects incomplete or malformed provider records.
- Ten-minute Laravel cache reduces repeated remote calls.
- Versioned fallback data keeps the assignment demonstrable offline.
- Blade escapes provider values before rendering.
- Fixed server-side endpoint avoids an arbitrary-URL SSRF surface.
- `Http::fake()` keeps automated tests deterministic and independent of the internet.

7. Advanced Code Evidence

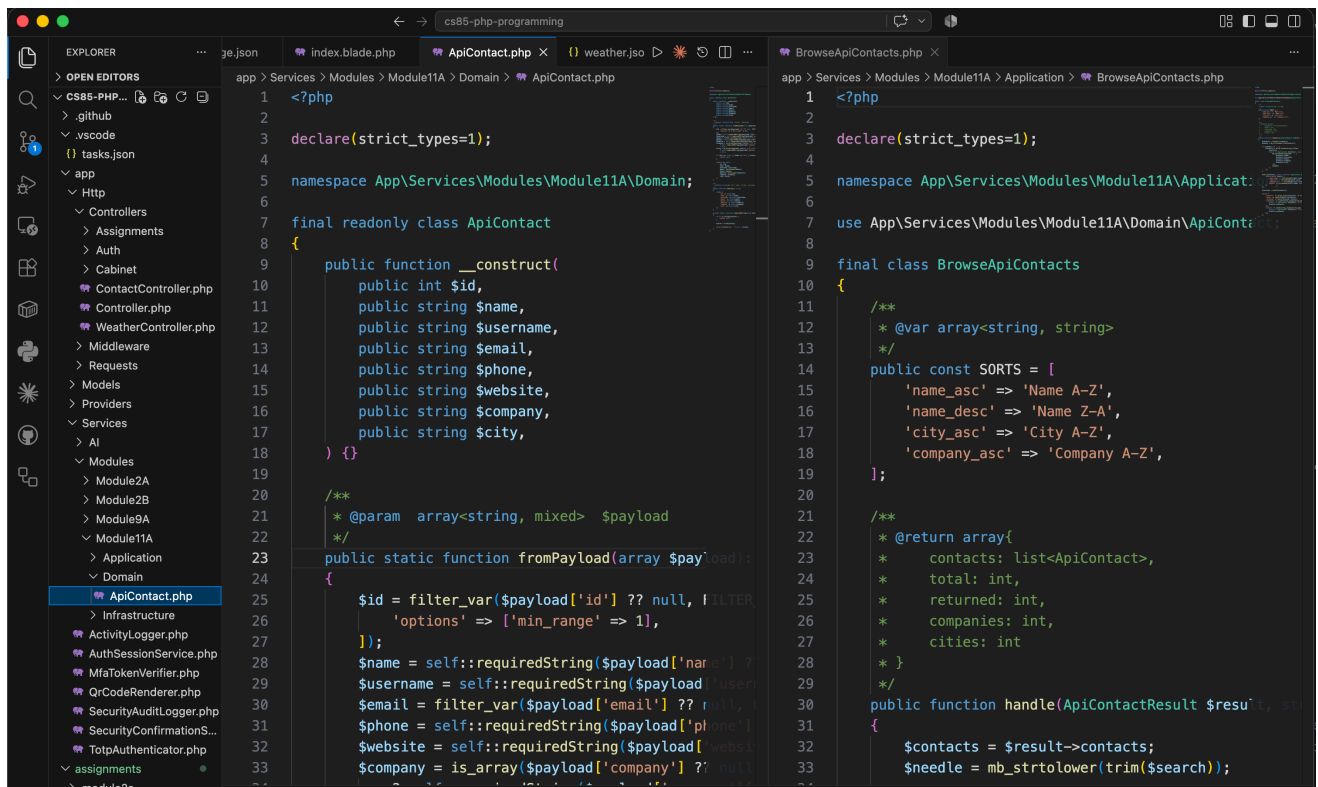


Figure 9. Domain and application layers

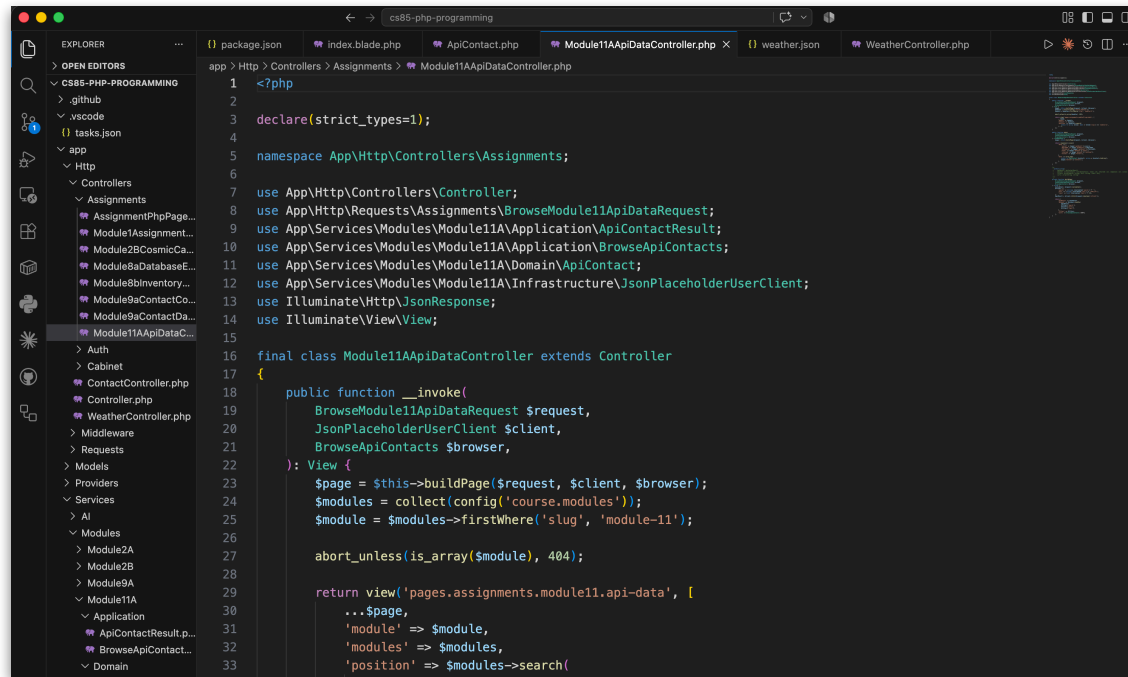


Figure 10. Infrastructure and presentation layers

8. Comparison and Reflection

Concern	Course-aligned track	Advanced implementation
Data source	Static weather.json file	Live JSONPlaceholder endpoint
Decode step	json_decode(\$json, true)	Laravel HTTP client response JSON
Data shape	Associative arrays	Typed ApiContact objects
Transformation	Optional alphabetical sort	Validated search, sort, limit, and statistics
Reliability	File is always local	Timeout, ten-minute cache, and local fallback
Output	Styled Blade weather table	Blade workbench plus normalized JSON endpoint
Purpose	Learn the JSON → PHP → Blade flow	Demonstrate production-oriented API boundaries

Reflection

The main assignment made the JSON lifecycle visible by keeping every step small: a known file, one controller, an associative array, and one Blade loop. This is the correct level for learning what JSON decoding actually does.

The advanced implementation showed why production API work needs additional boundaries. Remote data must be treated as untrusted, outages must be expected, and the presentation layer should not know provider-specific details. The DTO, application service, API client, cache, and fallback each solve a distinct problem.

Final takeaway: The course-aligned track proves the required JSON, controller, and Blade skills. The advanced track proves that the same foundation can evolve into a secure, testable, and resilient Laravel API integration.

Submission Checklist

- The /weather page is visible and shows all three forecast rows.
- weather.json is included from storage/app/private.
- WeatherController shows Storage::get() and json_decode(..., true).
- The Blade file shows @foreach and the four required columns.
- The optional conditional formatting and alphabetical sort are documented.
- The advanced page and normalized JSON endpoint are documented separately.
- README setup instructions and reflections are included.
- Automated tests pass before submission.
- All screenshot placeholders in this document are replaced with real screenshots.

```

•> php artisan test --filter=Module11.

[PASS] Tests\Feature\Module11AdvancedApiDataTest
✓ module 11 renders the advanced track with live api data 0.06s
✓ module 11 filters and sorts normalized contacts 0.01s
✓ module 11 json endpoint exposes the trusted contract 0.01s
✓ module 11 uses the versioned fallback when the api fails 0.01s
✓ module 11 reuses a successful cached response 0.01s
✓ module 11 rejects unknown query controls 0.01s
✓ module 11 course configuration registers the advanced view

[PASS] Tests\Feature\Module11WeatherAssignmentTest
✓ course aligned weather page decodes the required static json 0.01s
✓ course aligned weather page supports optional alphabetical sorting 0.01s
✓ course aligned files show the required storage decode and blade flow
✓ module 11 course configuration registers assignment 11a

Tests: 11 passed (75 assertions)
Duration: 0.24s

~/VSCode/cs85-php-programming main >6 ?1 >

```

Figure 11. Passing Module 11 test evidence

Project URLs and Files

Course-aligned page: <http://127.0.0.1:8000/weather>

Advanced page: <http://127.0.0.1:8000/roadmap/module-11>

Normalized JSON: <http://127.0.0.1:8000/roadmap/module-11/data>

Documentation: [assignments/module11a/README.md](#)